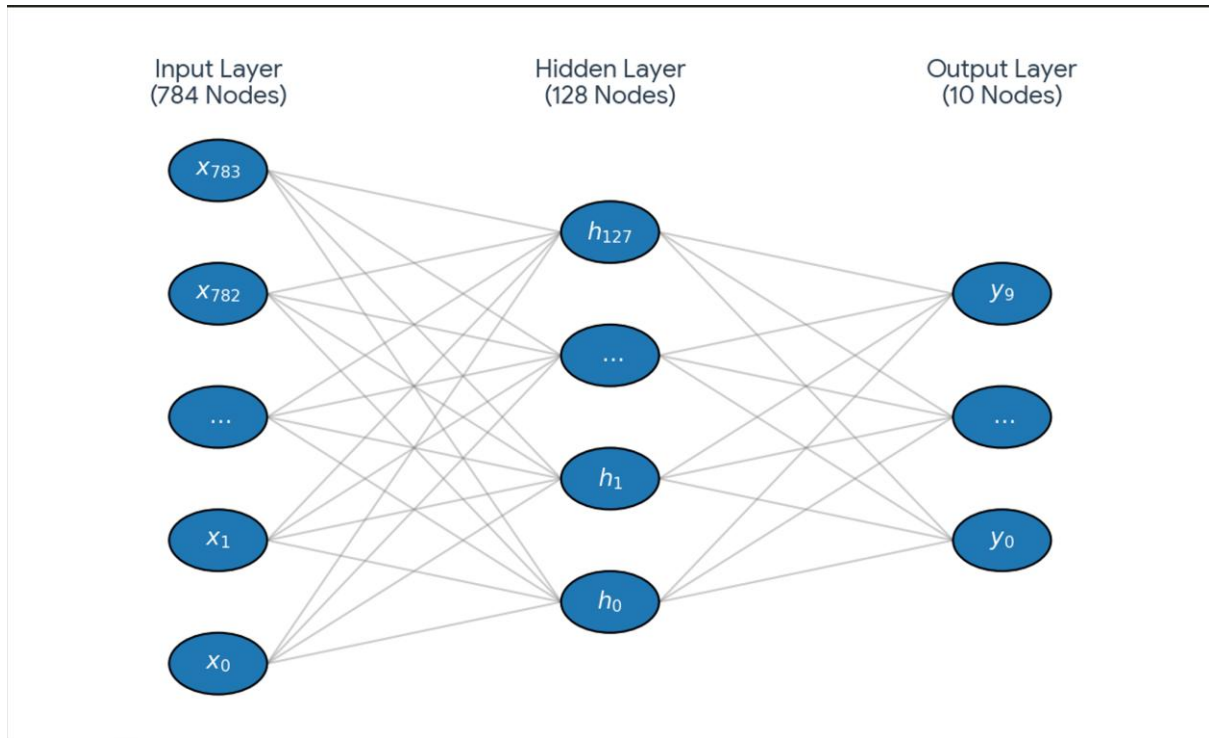


MLP 신경망 구조



입력노드에서 입력 이미지는 28x28임.

각 layer의 크기는 다음 표를 참조.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 10)	1290

입력 이미지는 data가 8비트 양의 정수임.

L1 노드 계산

784 channel (입력)과 128 filter (출력)로 구성.

$$A_k = R \left(\sum_{i=0}^{27} \sum_{j=0}^{27} I_{ij} * w_{(i*28+j)k}^1 + b_k^1 \right)$$

여기서 $i=0,...,27, j=0,...,27, k=0,...,127$ 임.

A_k 는 L1의 k 번째 filter임.

I_{ij} 은 입력 이미지의 i 번째 height, j 번째 width임.

$w_{(i*28+j)k}^1$ 은 $(i*28+j)$ 번째 channel과 k 번째 filter 사이의 weight임

b_k^1 는 k 번째 filter의 bias 값임.

$R()$ 은 ReLU함수임.

$$R(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

기본적으로 A_k, w_{kmn}^1 와 b_k^1 는 32비트 실수임. I_{ij} 는 0~255사이의 양의 정수임. 본 강의에서는 I_{ij} 를 256으로 나눈 값을 사용함.

L2 출력노드 계산

128개 channel과 10개의 filter로 구성

$$O_k = \sum_{i=0}^{127} A_i * w_{ik}^2 + b_k^2$$

여기서 $k=0,...,9$ 임.

O_k 는 L2의 k 번째 filter

A_i 는 L2의 i 번째 channel

w_{ik}^2 는 i 번째 channel과 k 번째 filter 사이의 weight임.

b_k^2 는 k 번째 filter의 bias값임.

Backpropagation 학습을 위해 출력노드의 activation은 softmax() 계산을 함.

이 강의에서는 학습이 끝난 MLP를 구현하므로 softmax() 값을 구할 필요가 없음.

대신에 음의 정수를 제거해서 FPGA 구현이 쉽도록 activation함수로 ReLU()를 사용함

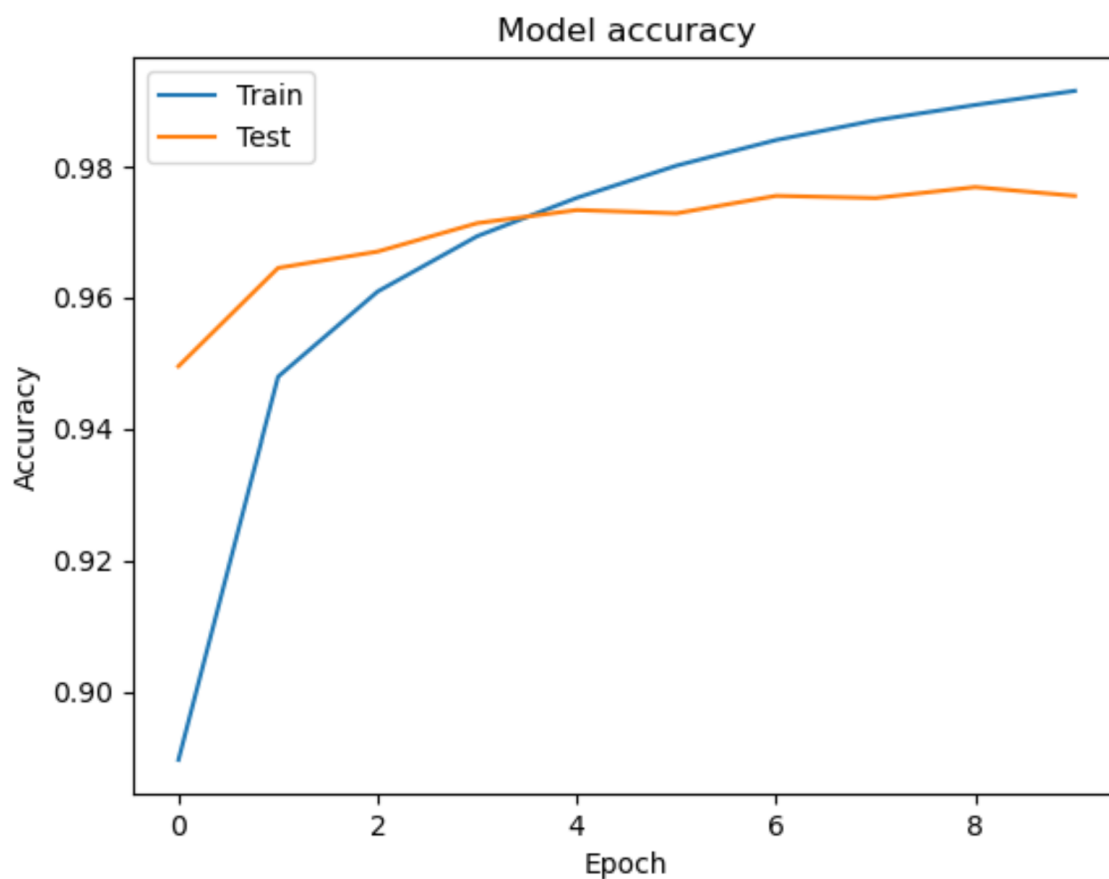
최대값 계산은 $\text{ReLU}(O_k)$, $k=0,...,9$, 값 중 최대값을 구함.

Quantization aware training

weight 및 bias를 8비트, 노드 A는 8비트, 노드 O는 12비트로 quantization 함.

weight 및 bias는 부호 1비트와, 소수점 이하 7비트를 갖는 이진수로 quantization을 하고 128을 곱했을 때 -127 ~ 127의 대칭적인 값을 갖도록 제한하고, 노드 A와 노드 B는 각각 0 또는 양의 정수를 갖도록 제한해서 학습을 시켰음.

MNIST 테스트 데이터로 정확도 97 % 정도임.



Quantization 방법에 따라 weight, bias, node 등의 비트수를 더 줄일 수 있음.

구현용 C 코드

```
#include <stdlib.h>

#include <stdio.h>

int main(){
    float in[784] = {          // saved in float format. 8 bit, 28*28, c only
#include "layeroutput/test_input.h"
    };

    // int in[784];          // 8bit x 784 depth. verilog only

    //
    // node: RAM, unsigned
    //
    int I[784];              // L0. height x width          8 bits,  28*28
    int A[128];              // L1                          8 bits
    int O[10];               // L2                      12 bits

    //
    // weight, bias: ROM, 8 bits, signed
    //
    int w1[100352] = {      // L1. input_channels x num_filters, 784*128
#include "weightoutput/w_dense_1.h"
    };
    int b1[128] = {          // L1. num_filters
#include "weightoutput/b_dense_1.h"
    };
    int w2[1280] = {         // L2. input_channels x num_filters, 128*10
#include "weightoutput/w_dense_2.h"
    };
```

```

int b2[10] = {          // L2. num_filters
#include "weightoutput/b_dense_2.h"
};

int max_value;          // 12 bit

//
// loop index
//
int i, j, k;             // 8, 6, 8 bit

//
// test result
//
int result;              // 4 bit

// round and clip
int rem;                  // 2 bit, signed

//
// test
//

int a0;                   // 10bit

// load input
a0 = 0;
for (i=0; i<=27; i++){
    for (j=0; j<=27; j++){
        l[a0] = (int)(in[a0]*256); // c only, l[i][j] = in[i][j]*256
    }
}

```

```

//l[a0] = in[a0];          // verilog only,  in[] saved in 8bit int

// test
printf("l[%d][%d] = %d\n", i, j, l[a0]);
// test

a0++;
}
}

//
// L1
//

int a_w1;    // 20 bit, signed
int a1;      // 17 bit

// L1
for (k=0; k<=127; k++){          // k: num_filters
    a_w1 = 0;
    for (i=0; i<=27; i++){        // i: height
        for (j=0; j<=27; j++){    // j: width. input_channels = i*28 + j
            a0 = i*28 + j;
            a1 = (i*28 + j) * 128 + k;    // (i*28+j)*128+k : 17bit
            a_w1 += (0<<8 | l[a0]) * w1[a1];    // a_w1 += (0<<8 | l[i][j]) * w1[i*28 + j][k];
        }
    }
}

// test
printf("k = %d, aw_1 = %d\n", k, a_w1);
// test

```

```

a_w1 += b1[k] * 256;

// relu activation
if (a_w1 < 0){
    a_w1 = 0;
}

// round and clip
// l: 8bit, w, b:7bit excluding sign bit.  a_w1 = l*w + b. a_w1: at least 15bits.
// A: 8bit. remove lower 7bits of a_w1
rem = a_w1 & 0x40;
rem = rem >> 6;
a_w1 = (a_w1 >> 7) + rem;
if (a_w1 >= 255){ // clipping
    A[k] = 255;
}
else {
    A[k] = a_w1 & 0xff; //8bit
}

// test
printf("A[%d] = %d\n", k, A[k]);
// test

} //k

//
// L2
//

```



```

int a_w2;  // 20 bit, signed
int a2;    // 11 bit

for (k=0; k<=9; k++){          // k: num_filters
    a_w2 = 0;
    for (i=0; i<=127; i++){    // i: input_channels
        a2 = i*10 + k;         // i*10+k: 11bit
        a_w2 += (0<<8 | A[i]) * w2[a2];    // a_w2 += (0<<8 | A[i]) * w2[i][k];
    }

    // test
    printf("k = %d, aw_2 = %d\n", k, a_w2);
    // test

    a_w2 += b2[k] * 256;

    // relu activation
    if (a_w2 < 0){
        a_w2 = 0;
    }

    // truncate and clip
    a_w2 = (a_w2 >> 7);

    if (a_w2 >= 2047){ // clipping
        O[k] = 2047;
    }
    else {
        O[k] = a_w2 & 0x3ff;    //12bit
    }
}

```

```

    // test
    printf("O[%d] = %d\n", k, O[k]);
    // test

}

// find max
result = 0;
max_value = O[0];
for (k=1; k<=9; k++){
    if (max_value < O[k]){
        max_value = O[k];
        result = k;
    }
}

//test
printf("result = %d\n", result);

return 0;

}

```

배열변수: RAM 또는 ROM

변수: 레지스터로 모델링

Xilinx FPGA에서는 메모리가 2차원 배열까지만 구현 가능. 구현이 복잡함.

RAM과 ROM은 1차원 메모리로 구현. 메모리의 data width, depth를 결정해야 함.

변수: width를 결정해야 함.

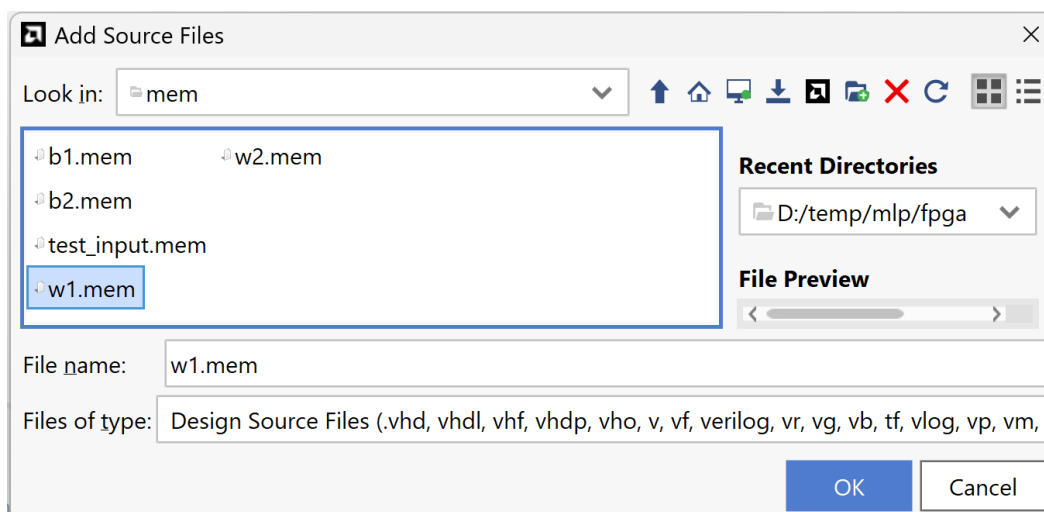
IP catalog를 사용한 RAM, ROM IP 만들기

project part:

xc7a200tsbg484-3 선택

w1_rom 생성하기

Add source → add or create design sources → add files → w1.mem 선택



다음 부분 체크 확인

- ☒ Scan and add RTL include files into project
- ☒ Copy sources into project
- ☒ Add sources from subdirectories

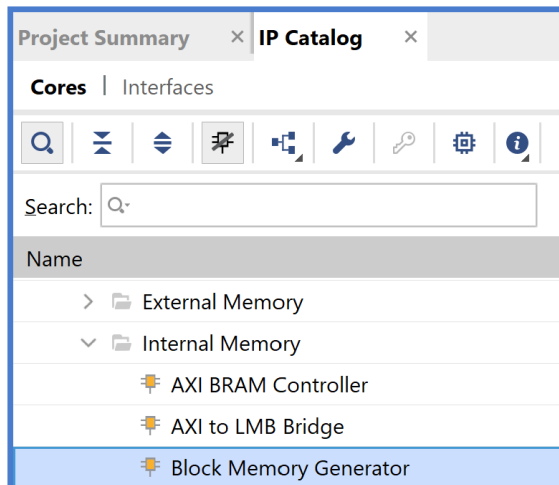
Finish 버튼 클릭

2. IP generator를 사용한 bram IP 추가

IP Catalog 클릭

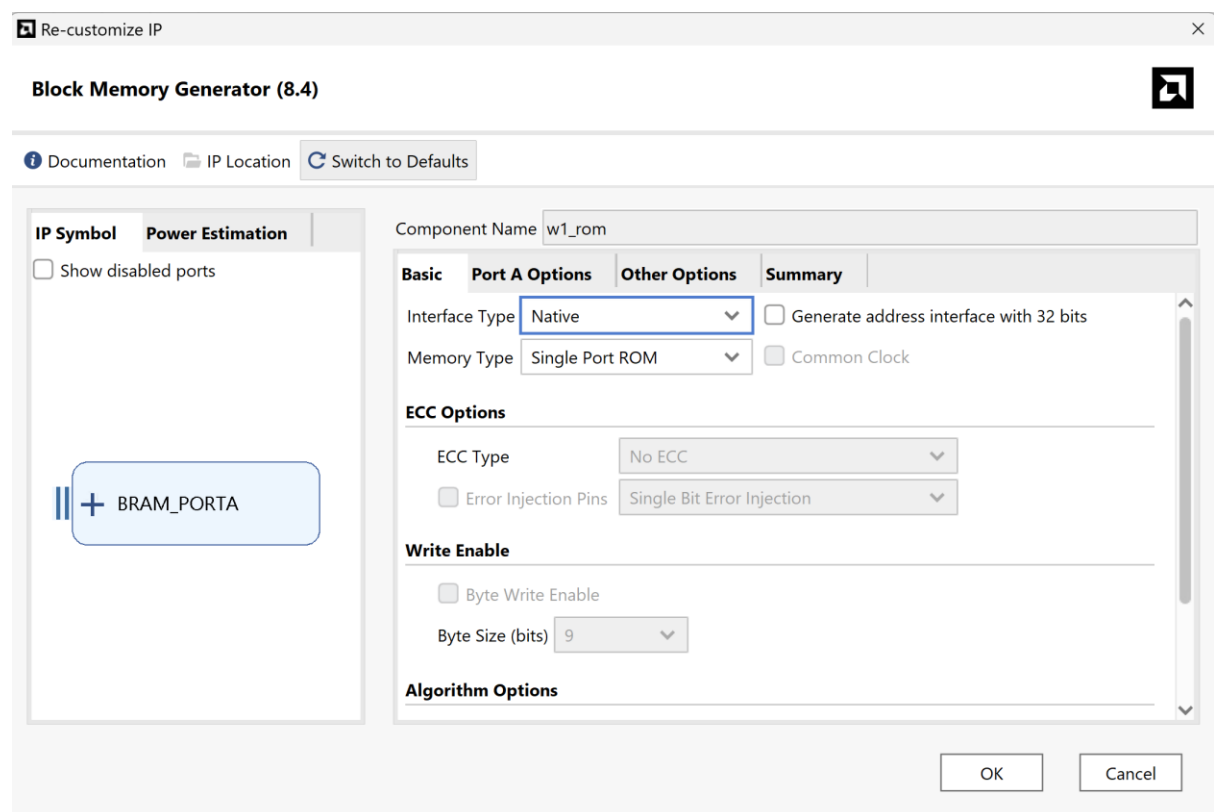
 IP Catalog

Block memory generator 선택



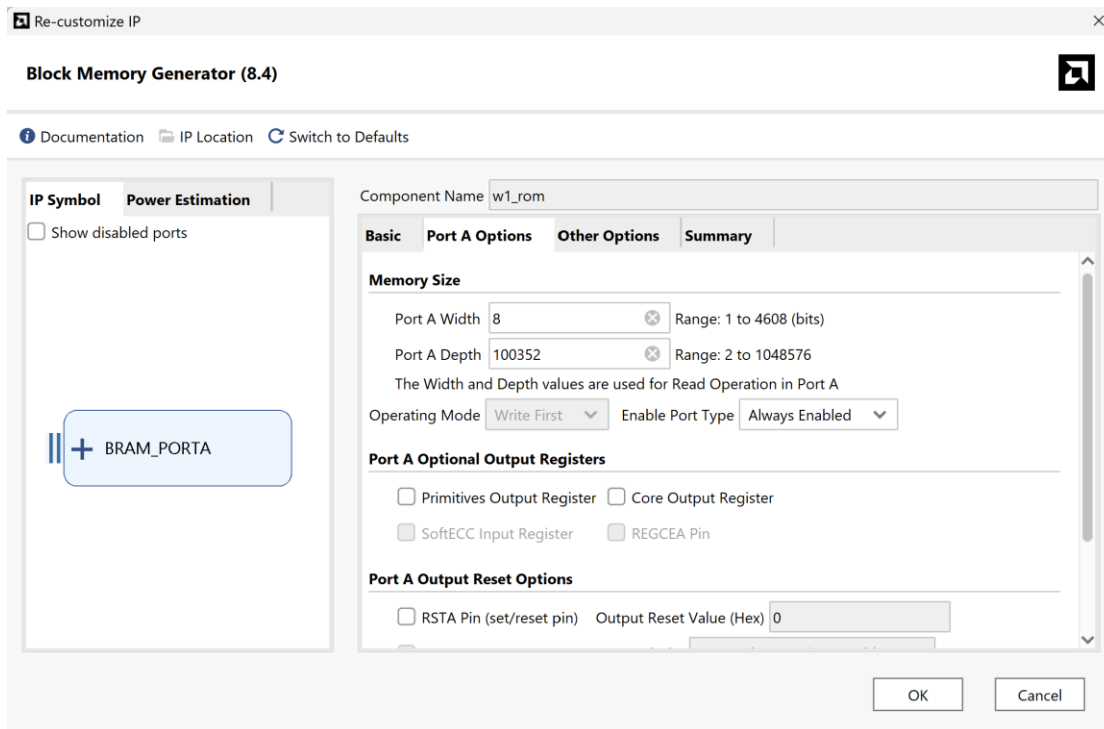
component name은 w1_rom

native interface, single port ROM 선택

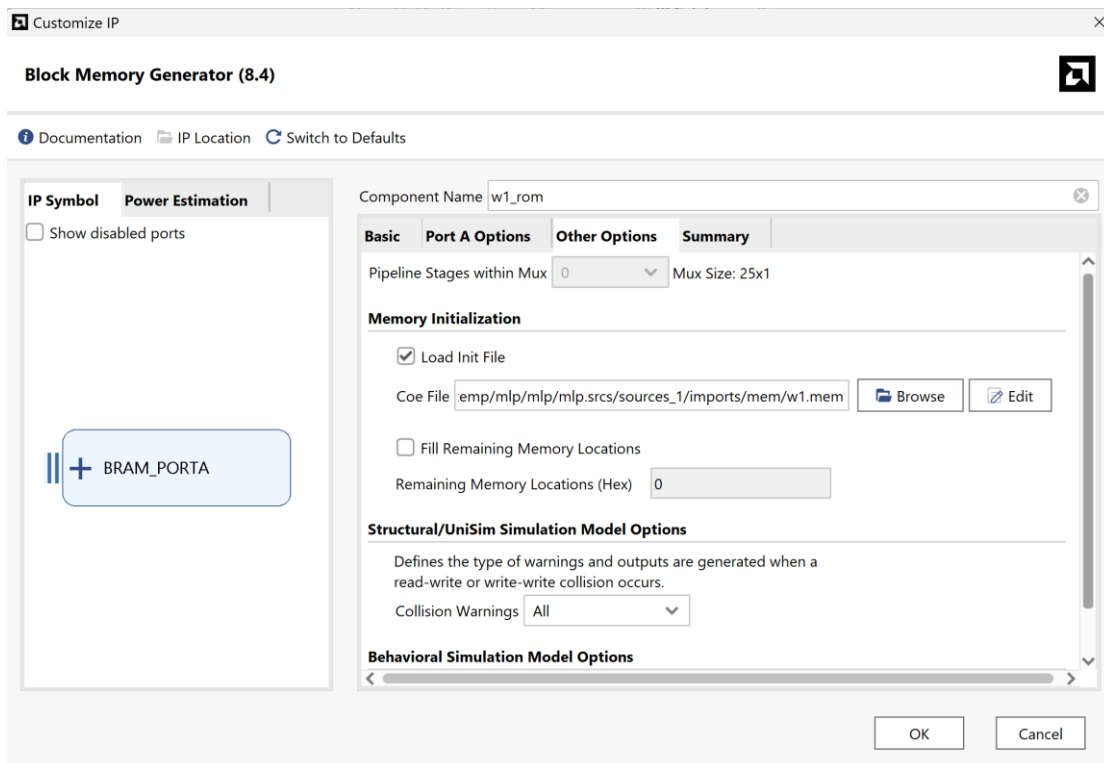


bram 사양 선택: width 8, depth 100352, always enabled

output register는 어떤 옵션도 선택하지 않음



초기화 파일로 w1.mem 지정. imports → mem (없을 수도 있음) → w1.mem (확장자 all files로 변경)



b1_rom 생성하기

비슷하게 b1.mem 메모리 파일 추가함.

component name은 b1_rom

native interface, single port ROM 선택

bram 사양 선택: width 8, depth 128, always enabled

output register는 어떤 옵션도 선택하지 않음

초기화 파일로 b1.mem 지정

w2_rom 생성하기

비슷하게 w2.mem 메모리 파일 추가함.

component name은 w2_rom

native interface, single port ROM 선택

bram 사양 선택: width 8, depth 1280, always enabled

output register는 어떤 옵션도 선택하지 않음

초기화 파일로 w2.mem 지정

b2_rom 생성하기

비슷하게 b2.mem 메모리 파일 추가함.

component name은 b2_rom

native interface, single port ROM 선택

bram 사양 선택: width 8, depth 10, always enabled

output register는 어떤 옵션도 선택하지 않음

초기화 파일로 b2.mem 지정

test_rom 생성하기

비슷하게 test_input.mem 메모리 파일 추가함.

component name은 test_rom

native interface, single port ROM 선택

bram 사양 선택: width 8, depth 784, always enabled

output register는 어떤 옵션도 선택하지 않음

초기화 파일로 test_input.mem 지정

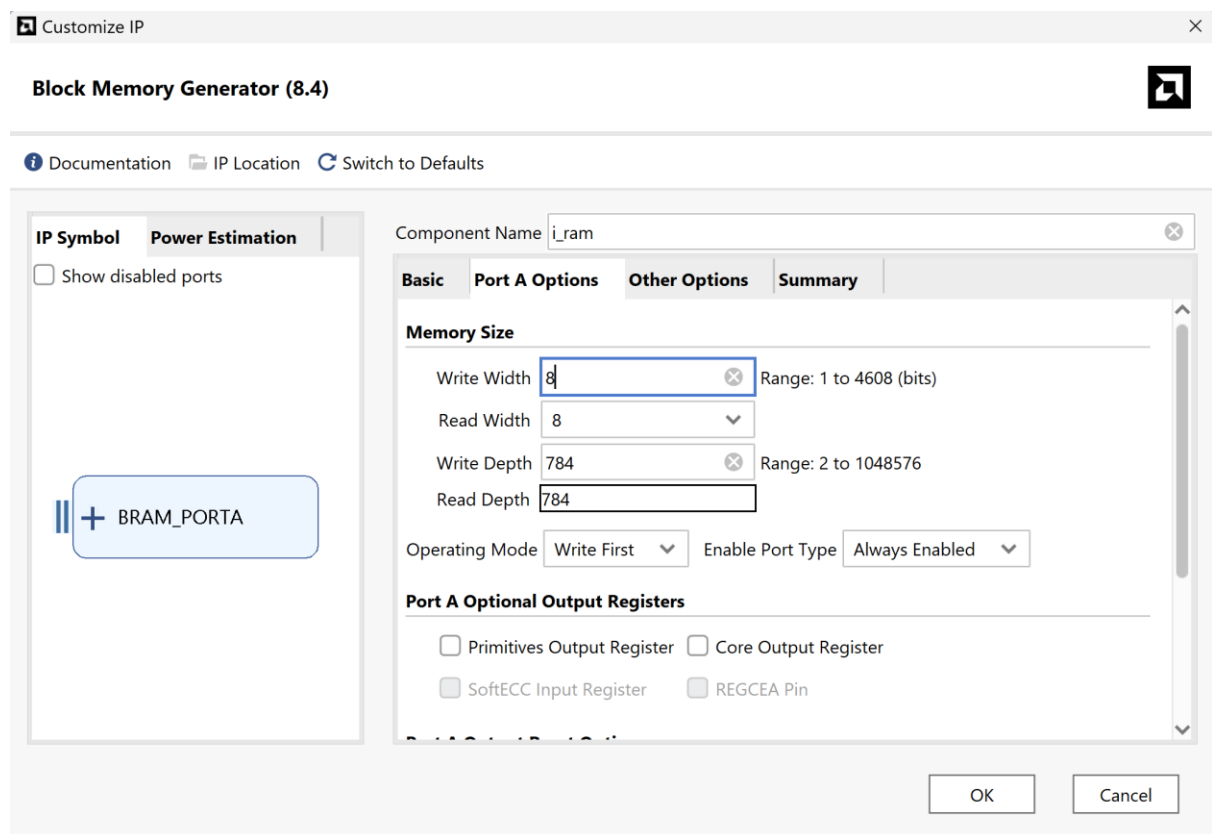
i_ram 생성하기

component name은 i_ram

native interface, single port RAM 선택

bram 사양 선택: width 8, depth 784, write first, always enabled

output register는 어떤 옵션도 선택하지 않음



a_ram 생성하기

component name은 a_ram

native interface, single port RAM 선택

bram 사양 선택: width 8, depth 128, write first, always enabled

output register는 어떤 옵션도 선택하지 않음

o_ram 생성하기

component name은 o_ram

native interface, single port RAM 선택

bram 사양 선택: width 12, depth 10, write first, always enabled

output register는 어떤 옵션도 선택하지 않음

Signed vs. unsigned

```
reg signed [7:0] a;
```

```
reg [7:0] b;
```

```
initial
```

```
begin
```

```
result = a;           //Signed
```

```
result = a * a;       //Signed
```

```
result = a * 10;      //Signed
```

```
result = $unsigned(a); //Unsigned
```

```
result = a[0];        //Unsigned
```

```
result = a[7:0];      //Unsigned
```

```
result = {a,a};       //Unsigned
```

```
result = {10{a}};     //Unsigned
```

```
result = a + b;       //Unsigned
```

```
result = a * b;       //Unsigned
```

```
end
```

```
reg      [3:0] a; // Unsigned
```

```
wire signed [4:0] signed_a;
```

```
wire signed [3:0] signed_b;
```

```
assign a = 8;
```

```
assign signed_a = a; // Convert to signed. signed_a = $signed({1'b0, a}) = +8
```

```
assign signed_b = a; //convert to signed. signed_b = -8;
```

mlp.v

```
module mlp
(
    input          clk,
    input          reset,
    output reg [9:0] in_addr,
    input [7:0]     in_rdata,
    output reg [3:0] result
);

//
// memory
//

// RAM, unsigned
reg [9:0]      i_addr;
reg [7:0]      i_wdata;
wire [7:0]     i_rdata;
reg           i_we;

reg [6:0]      a_addr;
reg [7:0]      a_wdata;
wire [7:0]     a_rdata;
reg           a_we;

reg [3:0]      o_addr;
reg [11:0]     o_wdata;
wire [11:0]    o_rdata;
reg           o_we;

// ROM
reg [16:0]     w1_addr;
wire signed [7:0] w1_rdata;

reg [6:0]      b1_addr;
wire signed [7:0] b1_rdata;
```

```

reg [10:0]      w2_addr;
wire signed [7:0] w2_rdata;

reg [3:0]      b2_addr;
wire signed [7:0] b2_rdata;

//
// register
//

// output node
reg [11:0]      max_value;
reg [11:0]      nmax_value;

// loop index
reg [7:0]      i;
reg [7:0]      ni;
reg [5:0]      j;
reg [5:0]      nj;
reg [7:0]      k;
reg [7:0]      nk;

// test result
//reg [3:0]      result;
reg [3:0]      nresult;

// round and clip
reg signed [1:0] rem;
reg signed [1:0] nrem;

// load input
reg [9:0]      a0;
reg [9:0]      na0;

// L1
reg signed [19:0] a_w1;
reg signed [19:0] na_w1;
reg [16:0]      a1;

```

```

reg [16:0]          na1;

// L2
reg signed [19:0]  a_w2;
reg signed [19:0]  na_w2;
reg [10:0]         a2;
reg [10:0]         na2;

//
// state
//
localparam idle0 = 0, load0 = 1, lay1_1 = 2, lay1_2 = 3, lay2_1 = 4,
             lay2_2 = 5, max0 = 6, max1 = 7, max2 = 8, wait0 = 9,
             load1 = 10, lay1_3 = 11;

reg [5:0]          state;
reg [5:0]          nstate;

//
// file
//
integer           fd;

//
//
// RAM instantiation
//
i_ram i_mlp (.clka(clk), .wea(i_we), .addra(i_addr), .dina(i_wdata),
             .douta(i_rdata));

a_ram a_mlp (.clka(clk), .wea(a_we), .addra(a_addr), .dina(a_wdata),
             .douta(a_rdata));

o_ram o_mlp (.clka(clk), .wea(o_we), .addra(o_addr), .dina(o_wdata),
             .douta(o_rdata));

```

```

//
// ROM instantiation
//
w1_rom w1_mlp (.clka(clk), .addra(w1_addr), .douta(w1_rdata));

b1_rom b1_mlp (.clka(clk), .addra(b1_addr), .douta(b1_rdata));

w2_rom w2_mlp (.clka(clk), .addra(w2_addr), .douta(w2_rdata));

b2_rom b2_mlp (.clka(clk), .addra(b2_addr), .douta(b2_rdata));

//
// FSM
//
always @ (reset, state, in_rdata, i_rdata, a_rdata, o_rdata,
          w1_rdata, b1_rdata, w2_rdata, b2_rdata,
          i, j, k, result, rem, a0, a_w1, a1, a_w2, a2, max_value
          ) begin

    // memory default
    in_addr = 0;
    i_addr  = 0;
    i_wdata = 0;
    i_we    = 0;
    a_addr  = 0;
    a_wdata = 0;
    a_we    = 0;
    o_addr  = 0;
    o_wdata = 0;
    o_we    = 0;
    w1_addr = 0;
    b1_addr = 0;
    w2_addr = 0;

```

```

b2_addr = 0;

// register default
nstate    = state;
nmax_value = max_value;
ni         = i;
nj         = j;
nk         = k;
nresult    = result;
nrem       = rem;
na0        = a0;
na_w1      = a_w1;
na1        = a1;
na_w2      = a_w2;
na2        = a2;

```

```

case (state)
  idle0: begin
    nstate = load0;
    na0 = 0;
    ni = 0;
    nj = 0;
    in_addr = 0;
  end // case: idle0

```

```

load0: begin
  i_wdata = in_rdata;
  i_addr = a0;
  i_we = 1'b1;

  na0 = a0 + 1;
  in_addr = na0;
  nstate = load0;

  nj = j + 1;
  if (nj == 28) begin
    nj = 0;

```

```

        ni = i + 1;
        if (ni == 28) begin
            ni = 0;
            nstate = load1;
        end
    end
end // case: load0

load1: begin
    nstate = lay1_1;
    nk = 0;
    na_w1 = 0;
    ni = 0;
    nj = 0;
    i_addr = 0;
    w1_addr = 0;
end // case: load1

//
// L1
//
lay1_1: begin
    na_w1 = a_w1 + $signed({1'b0, i_rdata}) * w1_rdata;

    nstate = lay1_1;
    nj = j + 1;
    if (nj == 28) begin
        nj = 0;
        ni = i + 1;
        if (ni == 28) begin
            ni = 0;
            nstate = lay1_2;
            b1_addr = k;
        end
    end
end

na0 = ni*28 + nj;
na1 = (ni*28 + nj)*128 + k;

```



```

    i_addr = na0;
    w1_addr = na1;
end // case: lay1_1

lay1_2: begin
    na_w1 = a_w1 + b1_rdata * 256;

    if (na_w1[19] == 1'b1) begin
        na_w1 = 0;
    end

    nrem = na_w1[6];
    na_w1 = na_w1[19:7] + nrem;

    if (na_w1 >= 255) begin
        a_wdata = 255;
    end
    else begin
        a_wdata = na_w1[7:0];
    end
    a_addr = k;
    a_we = 1'b1;

    nk = k + 1;
    if (nk == 128) begin
        nk = 0;
        nstate = lay1_3;
    end
    else begin
        na_w1 = 0;
        ni = 0;
        nj = 0;
        na0 = 0;
        na1 = nk;
        i_addr = na0;
        w1_addr = na1;
        nstate = lay1_1;
    end
end

```

```

        end
    end // case: lay1_2

    lay1_3: begin
        nk = 0;
        nstate = lay2_1;
        na_w2 = 0;
        ni = 0;
        na2 = 0;
        a_addr = 0;
        w2_addr = 0;
    end

    //
    // L2
    //
    lay2_1: begin
        na_w2 = a_w2 + $signed({1'b0, a_rdata}) * w2_rdata;

        nstate = lay2_1;
        ni = i + 1;
        if (ni == 128) begin
            ni = 0;
            nstate = lay2_2;
            b2_addr = k;
        end

        na2 = ni*10 + k;
        a_addr = ni;
        w2_addr = na2;
    end // case: lay2_1

    lay2_2: begin
        na_w2 = a_w2 + b2_rdata * 256;

        if (na_w2[19] == 1'b1) begin
            na_w2 = 0;
        end
    end

```

```

na_w2 = na_w2[19:7];

if (na_w2 >= 2047) begin
    o_wdata = 2047;
end
else begin
    o_wdata = na_w2[11:0];
end
o_addr = k;
o_we = 1'b1;

nk = k + 1;
if (nk == 10) begin
    nk = 0;
    nstate = max0;
end
else begin
    na_w2 = 0;
    ni = 0;
    na2 = nk;
    a_addr = ni;
    w2_addr = na2;
    nstate = lay2_1;
end
end // case: s2

//
// find max
//
max0: begin
    o_addr = 0;
    nstate = max1;
end

max1: begin
    nresult = 0;
    nmax_value = o_rdata;

```

```

    nk = 1;
    o_addr = nk;
    nstate = max2;
end

max2: begin
    if (max_value < o_rdata) begin
        nmax_value = o_rdata;
        nresult = k;
    end

    nstate = max2;
    nk = k + 1;
    if (nk == 10) begin
        nk = 0;
        nstate = wait0;
    end

    o_addr = nk;
end // case: max2

wait0: begin
    nstate = wait0;
end

default: begin
    nstate = wait0;
end

endcase

if (reset == 1) begin
    nstate      = idle0;
    nmax_value = 0;
    ni          = 0;
    nj          = 0;
    nk          = 0;
    nresult     = 0;

```

```

        nrem      = 0;
        na0       = 0;
        na_w1     = 0;
        na1       = 0;
        na_w2     = 0;
        na2       = 0;
    end
end

```

```

always @ (posedge clk)
begin
    state      <= nstate;
    max_value <= nmax_value;
    i          <= ni;
    j          <= nj;
    k          <= nk;
    result     <= nresult;
    rem        <= nrem;
    a0         <= na0;
    a_w1       <= na_w1;
    a1         <= na1;
    a_w2       <= na_w2;
    a2         <= na2;
end

```

```

//
// debug
//
// synthesis translate_off
always @ (posedge clk) begin

    fd = $fopen("aaa.txt", "a");

    if (state == load0) begin
        $fwrite(fd, "I[%0d][%0d] = %0d\n", i, j, i_wdata);
    end
    else if (state == lay1_2) begin

```

```

        $fwrite(fd, "k = %0d, a_w1 = %0d\n", k, a_w1);
        $fwrite(fd, "A[%0d] = %0d\n", a_addr, a_wdata);
    end
    else if (state == lay2_2) begin
        $fwrite(fd, "k = %0d, a_w2 = %0d\n", k, a_w2);
        $fwrite(fd, "O[%0d] = %0d\n", o_addr, o_wdata);
    end
    else if (state == max2) begin
        if (nk == 0) begin
            $fwrite(fd, "result = %0d\n", nresult);
        end
    end
end

if (reset == 1) begin
    fd = $fopen("aaa.txt", "w");
end

    $fclose(fd);
end
// synthesis translate_on

endmodule

```

testbench tb.v

```
module tb;
```

```
    reg        clk;
```

```
    reg        reset;
```

```
    wire [9:0] in_addr;
```

```
    wire [7:0] in_rdata;
```

```
    wire [3:0] result;
```

```
    //
```

```
    // ROM instantiation
```

```
    //
```

```
    test_rom test_mlp (.clka(clk), .addra(in_addr), .douta(in_rdata));
```

```
    //
```

```
    // mlp instantiation
```

```
    //
```

```
    mlp uut
```

```
    (
```

```
        .clk(clk),
```

```
        .reset(reset),
```

```
        .in_addr(in_addr),
```

```
        .in_rdata(in_rdata),
```

```
        .result(result)
```

```
    );
```

```
    // 100 MHz clock
```

```
    initial begin
```

```
        clk = 0;
```

```
        forever #5 clk = ~clk;
```

```
    end
```

```
    // reset
```

```
    initial begin
```

```
        reset = 1;
```

```
        #100  reset = 0;
    end

endmodule
```


cnn.xdc

```
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```